

Data Structures and Algorithms

Gabriel Kinshuk

Contents

1	Foundations	2
1.1	Fibonacci Sequence Algorithms	4
1.1.1	Fibonacci Number	4
1.1.2	Last digit of a Fibonacci number	5
1.1.3	Huge Fibonacci number mod m	6
1.1.4	Last digit of the sum of Fibonacci numbers	7
1.1.5	Last digit of the partial sum of Fibonacci numbers	7
1.1.6	Last digit of sum of squares of Fibonacci numbers	8
1.2	Greatest common divisor	10
1.3	Least common multiple	11
2	Data Structures	12
3	Algorithms on Graphs	13
4	Algorithms on Strings	14
5	Advanced Algorithms and Complexity	15
6	Genome Assembly (Capstone)	16

1 Foundations

Algorithm design draws on a small set of general strategies. Three of them account for most of the problems here: greedy choice, divide and conquer, and dynamic programming.

Most of these problems follow the same pattern. We begin with a correct brute-force method, identify why it is slow, and replace it with a method whose running time we can bound. The three paradigms are the recurring shapes those faster methods take.

Measuring running time

The wall-clock running time of a program depends on the machine, the language, and the compiler. Counting seconds is not portable across any of these. We instead count elementary operations, meaning arithmetic, comparisons, and array accesses, as a function of the input size n . By $T(n)$ we mean the largest such count over all inputs of size n . Fixing the worst case gives a guarantee that holds for every input, not only for convenient ones.

Counting operations removes the machine, but it still produces an expression like $T(n) = 5n + 3$ whose coefficients depend on the counting convention. Two observations let us discard them. The coefficients shift with the convention and the hardware, so they say nothing about the algorithm itself. And for large n the fastest-growing term dominates the rest, so the constant 3 is negligible against $5n$ and the factor 5 only rescales a quantity that is already linear in n . What remains after both observations is the order of growth, the form of $T(n)$ up to a constant factor and for large n .

Asymptotic notation makes the phrase “up to a constant factor and for large n ” precise. We work with functions $f, g: \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$, since operation counts are nonnegative.

Definition 1.1 (Big-O). $f(n) = O(g(n))$ if there exist constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

$$f(n) \leq c g(n) \quad \text{for all } n \geq n_0$$

The constant c absorbs the counting convention and the hardware. The threshold n_0 discards small inputs, where startup costs and lower-order terms can dominate but where the running time is irrelevant anyway. Read the definition as follows: past input size n_0 , the function f never exceeds a fixed multiple of g . So g bounds the growth of f from above.

Definition 1.2 (Big-Omega). $f(n) = \Omega(g(n))$ if there exist constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

$$c g(n) \leq f(n) \quad \text{for all } n \geq n_0$$

This reverses the inequality of Big-O. Past input size n_0 , the function f is at least a fixed multiple of g , so g bounds the growth of f from below.

Definition 1.3 (Big-Theta). $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$. Equivalently, there exist constants $c_1, c_2 > 0$ and $n_0 \in \mathbb{N}$ such that

$$c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \text{for all } n \geq n_0$$

A Big-Theta bound sandwiches f between two fixed multiples of g . When $f = \Theta(g)$ the two functions grow at the same rate up to constant factors, and g is a tight description of the growth of f . This is the bound we look for on an algorithm, since it locates the running time between two known multiples of a single function.

The three bounds share one pattern

Each claim is an existential statement about constants. To establish $f = O(g)$ we exhibit one pair (c, n_0) for which the inequality holds. To refute it we show that no such pair can exist. The constants need not be small or tight, only to exist. Big-O gives an upper bound, Big-Omega a lower bound, and Big-Theta both at once.

Theorem 1.1 (Limit test). *Let f and g be positive for large n , and suppose $L = \lim_{n \rightarrow \infty} f(n)/g(n)$ exists in $[0, \infty]$. If $0 < L < \infty$ then $f = \Theta(g)$. If $L = 0$ then $f = O(g)$ and $f \neq \Theta(g)$. If $L = \infty$ then $f = \Omega(g)$ and $f \neq \Theta(g)$.*

Proof. Suppose $0 < L < \infty$ and set $\varepsilon = L/2$. By the definition of the limit there is an n_0 with $|f(n)/g(n) - L| < L/2$ for all $n \geq n_0$. Then $L/2 < f(n)/g(n) < 3L/2$, and multiplying by $g(n) > 0$ gives $(L/2)g(n) < f(n) < (3L/2)g(n)$. These are the two Big-Theta bounds with $c_1 = L/2$ and $c_2 = 3L/2$. If $L = 0$, then for every $\varepsilon > 0$ we have $f(n) < \varepsilon g(n)$ past a threshold, which gives $f = O(g)$ with no matching lower bound. If $L = \infty$, then $f(n)/g(n)$ exceeds any fixed constant past a threshold, which gives $f = \Omega(g)$ with no matching upper bound. $\square$

Example 1.1. Let $f(n) = 3n^2 + 5n + 2$ and $g(n) = n^2$. For $n \geq 1$,

$$3n^2 \leq 3n^2 + 5n + 2 \leq 3n^2 + 5n^2 + 2n^2 = 10n^2$$

The lower bound holds because the dropped terms $5n + 2$ are nonnegative. The upper bound replaces each lower-order term by a multiple of n^2 , valid once $n \geq 1$. So $c_1 = 3$, $c_2 = 10$, and $n_0 = 1$ satisfy the Big-Theta definition, and $f = \Theta(n^2)$. The limit test confirms it faster: $f(n)/g(n) = 3 + 5/n + 2/n^2 \rightarrow 3$.

Example 1.2. $n = O(n^2)$, since $n \leq n^2$ for $n \geq 1$. The reverse fails: $n \neq \Omega(n^2)$, because $cn^2 \leq n$ rearranges to $cn \leq 1$, which breaks once $n > 1/c$, for any fixed $c > 0$. Hence $n \neq \Theta(n^2)$. An upper bound alone does not force a matching rate.

Remark 1.1. The equality in $f(n) = O(g(n))$ is one-directional. It means that f belongs to the set $O(g)$, so the equals sign reads as “is”. We never write $O(g(n)) = f(n)$.

Remark 1.2. Big-Omega describes a function, so it applies to whichever running time we name. Stating that an algorithm’s worst-case time is $\Omega(n \log n)$ lower-bounds the worst case. It does not describe the best case, and the two are often different.

Growth hierarchy

For large n , the common running times are ordered by growth,

$$1 \prec \log n \prec \sqrt{n} \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n \prec n!$$

where $a \prec b$ means $a = O(b)$ and $a \neq \Theta(b)$. A polynomial-time algorithm runs in $O(n^k)$ for some fixed k . An exponential one runs in 2^n or worse and leaves the range of direct computation near $n = 40$, where 2^n exceeds 10^{12} .

The brute-force solution to many of these problems is exponential, so the aim throughout is a polynomial-time method with a provable bound.

The three paradigms

Greedy algorithms build a solution one step at a time. At each step we make the choice that looks best in isolation and never revise it. The work is in proving that this local choice, called a safe move, is consistent with some globally optimal solution. When a safe move exists the algorithm is usually a single pass or a sort followed by a pass.

Divide and conquer splits an instance into smaller instances of the same problem, solves those recursively, and combines their answers. The running time follows from a recurrence that weighs the cost of splitting and combining against the cost of the subproblems. Binary search, merge sort, and the inversion-counting and closest-pair problems all take this form.

Dynamic programming applies when the subproblems overlap, so the same smaller instance is needed many times. We solve each subproblem once, store its answer in a table, and build larger answers from smaller ones. Reusing stored answers turns an exponential recursion into a polynomial table fill.

1.1 Fibonacci Sequence Algorithms

The n th number in the Fibonacci sequence follows:

$$F(n) = F(n-1) + F(n-2) \quad F(0) = 0 \quad F(1) = 1$$

We can then consider a few problems using this sequence.

1.1.1 Fibonacci Number

Suppose we wanted to write a simple program to calculate $F(n)$. First we consider the recursive approach, which most closely resembles the formal definition.

```
1 # Recursive
2 def fib(n):
3     if n <= 1:
4         return n
5
6     return fib(n-1) + fib(n-2)
```

This recursive form runs in $O(\varphi^n)$ time and $O(n)$ stack depth, where φ is the golden ratio. Two separate limits make it unusable. The time is exponential because each call recomputes the same subcases, so for n in the low hundreds the computation takes longer than is feasible. The stack depth is linear because the leftmost chain of calls descends n levels before any of them returns, so in Python it raises `RecursionError` once n passes the recursion limit near 1000. Recording each subcase once removes the recomputation. We can do this with a simple `for` loop or with memoization, meaning we store computed values for $O(1)$ lookup:

```

1 # For loop
2 def fib(n):
3     prev, current = 0, 1
4
5     for _ in range(n - 1):
6         prev, current = current, prev + current
7
8     return current

```

```

1 # Memoization
2 def fib(n):
3     memo = {0:0, 1:1}
4
5     def fib_helper(n):
6         if n not in memo:
7             memo[n] = fib_helper(n-1) + fib_helper(n-2)
8
9         return memo[n]
10
11    return fib_helper(n)

```

Both versions run in $O(n)$ time. The `for` loop keeps two values, so its space is $O(1)$. Memoization keeps all n values, so its space is $O(n)$, and it adds function-call and dictionary overhead. It also recurses $O(n)$ deep, so like the naive version it raises `RecursionError` for large n , while the loop does not. We still show it because it generalizes to problems where the dependency between subcases is not a simple sequential chain.

1.1.2 Last digit of a Fibonacci number

Again, we can solve this naively, first calling `fib(n)` and then taking the last digit:

```

1 def fib_last_digit(n):
2     return fib(n) % 10

```

This is correct but slow for large n . It builds the full value of $F(n)$, which has on the order of $n/5$ digits, so each addition scales with the digit count rather than being constant.

Since we only want the last digit, we can take the modulus *inside* the loop. The last digit of a sum depends only on the last digits of the addends, since

$$(a + b) \bmod 10 = ((a \bmod 10) + (b \bmod 10)) \bmod 10$$

so reducing at every step gives the same units digit as reducing at the end. Both variables then stay in $\{0, \dots, 9\}$, so each addition is $O(1)$ and the routine runs in $O(n)$ time and $O(1)$ space:

```

1 def fib_last_digit(n):
2     if n <= 1:
3         return n
4
5     previous = 0
6     current = 1
7
8     for _ in range(n - 1):
9         previous, current = current, (previous + current) % 10
10
11    return current

```

1.1.3 Huge Fibonacci number mod m

Next we consider $F(n) \bmod m$ for very large n . We have already established that computing a giant Fibonacci number is unwieldy as n grows, even when each step is a single addition. Reducing mod m inside the loop keeps every value below m , so each addition is $O(1)$, but the loop still runs n times. For n up to 10^{18} even $O(n)$ is unacceptable, so we need a strategy whose cost does not grow with n .

We use the Pisano period: the sequence $F(n) \bmod m$ is periodic with period $\pi(m)$. A period must exist by the pigeonhole principle. Each term reduces to a residue, meaning a remainder in $\{0, 1, \dots, m-1\}$. There are only m^2 ordered pairs of consecutive residues, so among the first $m^2 + 1$ pairs two must coincide.

The cycle returns to the start rather than to some later pair. The recurrence is also invertible mod m , since $F(k-1) = F(k+1) - F(k)$, so no pair has two different predecessors. The first repeated pair is therefore the seed pair $(0, 1)$, which makes the sequence periodic from the beginning.

Knowing the sequence begins with 0 and 1, we track it until those two values reappear consecutively. The loop is bounded by m^2 steps. Once we have the period, we reduce n modulo it and return the stored value at that index. If $m \ll n$ then the resulting $O(m^2)$ time will be much better than $O(n)$:

```

1 def fib_huge_array(n: int, m: int) -> int:
2     if n <= 1:
3         return n
4
5     sequence = [0, 1]
6
7     for _ in range(m * m):
8         next_val = (sequence[-1] + sequence[-2]) % m
9         sequence.append(next_val)
10
11    if sequence[-2] == 0 and sequence[-1] == 1:
12        period_length = len(sequence) - 2
13        return sequence[n % period_length]
14
15    return -1

```

1.1.4 Last digit of the sum of Fibonacci numbers

We now want the sum of the first $n + 1$ Fibonacci numbers, which we call $S(n)$:

$$S(n) = \sum_{i=0}^n F(i) = F(0) + F(1) + F(2) + \cdots + F(n)$$

Rather than add $n + 1$ terms, we look for a closed form. The recurrence $F(i + 2) = F(i + 1) + F(i)$ rearranges to

$$F(i) = F(i + 2) - F(i + 1)$$

so each term in the sum is a difference of two consecutive Fibonacci numbers. Writing every summand this way makes the sum telescope: each $F(i + 1)$ cancels the $F(i + 2)$ of the term before it, leaving only the outermost pieces. Taking $S(4)$ as a check:

$$\begin{aligned} S(4) &= F(0) + F(1) + F(2) + F(3) + F(4) \\ &= (F(2) - F(1)) + (F(3) - F(2)) + (F(4) - F(3)) + (F(5) - F(4)) + (F(6) - F(5)) \\ &= F(6) - F(1) \\ &= F(6) - 1 \end{aligned}$$

The surviving terms are $F(n + 2)$ from the last summand and $-F(1) = -1$ from the first, giving

$$S(n) = F(n + 2) - 1$$

For the last digit we reduce mod 10. Since $1 \bmod 10 = 1$,

$$S(n) \bmod 10 = (F(n + 2) \bmod 10 - 1) \bmod 10$$

The outer reduction matters because $F(n + 2) \bmod 10$ can be 0, which makes the subtraction -1 , and the reduction returns it to 9. This reuses the last-digit routine from before:

```
1 def sum_fib_last_digit(n):  
2     return (fib_last_digit(n + 2) - 1) % 10
```

The identity replaces the running sum with a single last-digit computation. Both are $O(n)$ time, so this is not an asymptotic improvement. What it removes is the accumulation: the naive sum carries a running total and adds to it at every step, whereas the identity computes one Fibonacci number and subtracts 1. The routine is `fib_last_digit`, which is $O(n)$ time and $O(1)$ space.

1.1.5 Last digit of the partial sum of Fibonacci numbers

Now we want to design a program to compute the last digit of the sum of Fibonacci numbers from integers m to n ($m < n$), which we'll call $P(m, n)$:

$$P(m, n) = \left(\sum_{i=m}^n F(i) \right) \bmod 10$$

For example, $P(3, 7)$:

$$\begin{aligned} P(3, 7) &= (F(3) + F(4) + F(5) + F(6) + F(7)) \bmod 10 \\ &= (2 + 3 + 5 + 8 + 13) \bmod 10 \\ &= 31 \bmod 10 = 1 \end{aligned}$$

We extend the approach from the previous problem. The full-sum identity is $S(k) = \sum_{i=0}^k F(i) = F(k+2) - 1$. A sum that starts at m is the full sum up to n with everything below m removed, which is $S(n) - S(m-1)$:

$$\begin{aligned} P(m, n) &= S(n) - S(m-1) \\ &= (F(n+2) - 1) - (F(m+1) - 1) \\ &= F(n+2) - F(m+1) \end{aligned}$$

Reducing mod 10:

$$P(m, n) = (F(n+2) \bmod 10 - F(m+1) \bmod 10) \bmod 10$$

The outer reduction keeps the result in $\{0, \dots, 9\}$, since the subtraction can go negative. We compute $F(n+2) \bmod 10$ with the two-variable loop and capture $F(m+1) \bmod 10$ as we pass it. After k iterations from $(0, 1)$, we have **current** = $F(k+1) \bmod 10$, so **current** holds $F(m+1)$ right after iteration m and $F(n+2)$ after iteration $n+1$:

```

1 def fib_last_digit_of_partial_sum(m, n):
2     previous, current = 0, 1          # F(0), F(1)
3     fib_m_plus_1 = 1                 # F(1), used when m == 0
4
5     for i in range(1, n + 2):
6         previous, current = current, (previous + current) % 10
7         if i == m:
8             fib_m_plus_1 = current    # F(m+1)
9
10    return (current - fib_m_plus_1) % 10    # (F(n+2) - F(m+1)) mod 10

```

1.1.6 Last digit of sum of squares of Fibonacci numbers

Now we want the last digit of the sum of the squares of Fibonacci numbers, which we'll call $Q(n)$ (for sQuare):

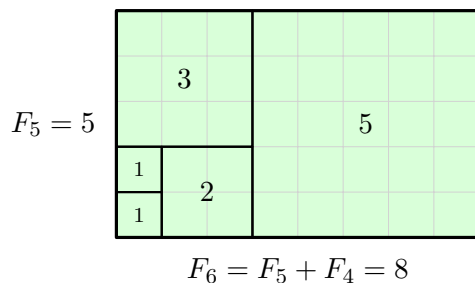
$$Q(n) = \left(\sum_{i=0}^n F(i)^2 \right) \bmod 10$$

Reduction commutes with multiplication, so for any product

$$(a \cdot b) \bmod 10 = ((a \bmod 10) \cdot (b \bmod 10)) \bmod 10$$

This lets us carry the two Fibonacci values already reduced and reduce once more after multiplying, which is what the final step below does.

The sum of squares of Fibonacci numbers up to n can be expressed as $F(n) \cdot F(n+1)$, illustrated here:



The squares of sides $F_1, \dots, F_5$ tile a rectangle of size $F_5 \times F_6$, so

$$\sum_{i=1}^5 F_i^2 = F_5 F_6 = 5 \cdot 8 = 40$$

Each new square of side F_k attaches along the full edge of the block built from $F_1, \dots, F_{k-1}$, because $F_k = F_{k-1} + F_{k-2}$ makes that edge exactly F_k long. Iterating gives the general identity

$$\sum_{i=0}^n F_i^2 = F_n F_{n+1}$$

where the $F_0 = 0$ term contributes no area.

To support a large n , we reuse the period search from the huge-Fibonacci problem. Recall that we would the last digits from $(0, 1)$ until the seed pair recurs, which must repeat within m^2 steps by pigeonhole. With the period in hand, $F(n) \bmod 10$ is the entry at index $n \bmod \text{period}$, and $F(n+1) \bmod 10$ the entry at $(n+1) \bmod \text{period}$. Their product, reduced once more, is $Q(n)$:

```

1 def fib_last_digit_of_squares(n):
2     m = 10
3
4     sequence = [0, 1] # F(0), F(1)
5     for _ in range(m * m):
6         sequence.append((sequence[-1] + sequence[-2]) % m)
7         if sequence[-2] == 0 and sequence[-1] == 1:
8             break # seed pair (0, 1) has returned
9     period = len(sequence) - 2 # drop the trailing seed pair
10
11     return (sequence[n % period] * sequence[(n + 1) % period]) % 10

```

The search builds one period in $O(m^2)$ time and space, but m is fixed at 10, so the routine is constant time and space.

1.2 Greatest common divisor

The efficient algorithm for computing $\gcd(a, b)$ comes from Euclid's *Elements* (c. 300 BCE) and is among the oldest algorithms still in use. It rests on division with remainder: for any a and $b > 0$ there are unique integers q and r with

$$a = bq + r \quad 0 \leq r < b$$

The bound $r < b$ is what makes the method shrink.

Any common divisor of a and b divides $r = a - bq$, so it is a common divisor of b and r . Conversely, any common divisor of b and r divides $a = bq + r$, so it is a common divisor of a and b . The two pairs therefore have identical sets of common divisors, and in particular the same greatest one:

$$\gcd(a, b) = \gcd(b, r)$$

We repeat with (b, r) in place of (a, b) . Each step replaces the second argument with a strictly smaller nonnegative remainder, so the remainders strictly decrease and reach 0 after finitely many steps. When $r = 0$ we have $\gcd(b, 0) = b$, since every integer divides 0, so the divisor at that final step is the answer. Because each step preserves the GCD, it equals $\gcd(a, b)$ for the original inputs.

For example with $a = 6$, $b = 4$:

$$\begin{aligned} 6 &= 1 \cdot 4 + 2 \\ 4 &= 2 \cdot 2 + 0 \\ \gcd(6, 4) &= \gcd(4, 2) = 2 \end{aligned}$$

For illustrative purposes, we also show the naive algorithm, which tests each integer from 2 up to the smaller of a or b :

```
1 def gcd_naive(a, b):
2     current_gcd = 1
3     for d in range(2, min(a, b) + 1):
4         if a % d == 0 and b % d == 0:
5             if d > current_gcd:
6                 current_gcd = d
7
8     return current_gcd
```

This runs in $O(\min(a, b))$ time. That is linear in the value of the input but exponential in its size, since a is written in about $\log a$ digits, which is why it is unusable for large inputs. Now the efficient algorithm:

```
1 def gcd(a, b):
2     while b != 0:
3         a, b = b, a % b
4
5     return a
```

This runs in $O(\log(\min(a, b)))$ divisions. The remainder at least halves every two steps: if $b \leq a/2$ then $a \bmod b < b \leq a/2$, and if $b > a/2$ then $a \bmod b = a - b < a/2$. Halving every two steps bounds the number of steps by $O(\log a)$. The input a need not exceed b , since when $a < b$ the first iteration computes $a \bmod b = a$ and swaps them.

1.3 Least common multiple

Lastly, we consider an algorithm for computing the least common multiple of two integers. Again we begin with a naive algorithm, which tests each multiple of the larger integer up to the product of the two:

```
1 def lcm_naive(a, b):
2     a, b = max(a, b), min(a, b)
3
4     for i in range(a, a * b + 1, a):
5         if i % b == 0:                                # by construction, i % a == 0
6             return i
```

Before finding a more efficient algorithm, we note some properties of the gcd and lcm. First, the gcd is the product of the shared terms in the prime factorizations of a and b . Take $\text{gcd}(32, 28)$, with factorizations

$$32 = 2^5 \quad 28 = 2^2 \cdot 7$$

The term common to both is $2^2 = 4$, which is the gcd. Writing a and b over a common set of primes (with zero exponents where a prime is absent):

$$a = \prod_i p_i^{\alpha_i}, \quad b = \prod_i p_i^{\beta_i} \quad \longrightarrow \quad \text{gcd}(a, b) = \prod_i p_i^{\min(\alpha_i, \beta_i)}$$

Each exponent is the min because a divisor of a cannot use more copies of p_i than a has.

Now consider the least common multiple, the smallest number that both a and b divide. A number m is divisible by a when, for every prime p_i , its exponent on p_i is at least α_i . For m to be divisible by both a and b , its exponent on each p_i must be at least α_i and at least β_i , hence at least $\max(\alpha_i, \beta_i)$. Taking exactly that maximum on every prime, and nothing more, gives the smallest such m :

$$\text{lcm}(a, b) = \prod_i p_i^{\max(\alpha_i, \beta_i)}$$

On each prime the two exponents satisfy $\min(\alpha_i, \beta_i) + \max(\alpha_i, \beta_i) = \alpha_i + \beta_i$, so multiplying the gcd and lcm recovers $a \cdot b$ prime by prime. Hence

$$\text{lcm}(a, b) = \frac{a \cdot b}{\text{gcd}(a, b)}$$

which reduces the lcm to one gcd computation. In code:

```
1 def lcm(a, b):
2     return a // gcd(a, b) * b
```

2 Data Structures

3 Algorithms on Graphs

4 Algorithms on Strings

5 Advanced Algorithms and Complexity

6 Genome Assembly (Capstone)